

Programação de Sistemas

Introdução à linguagem C

Marcelo de Gomensoro Malheiros



Linguagem C

- Projetada por Dennis Ritchie em 1972
- Nome derivado de B, que veio de BCPL
- Popularizada com o livro “The C Programming Language”, de Kernighan e Ritchie (K&R), em 1978
- Linguagem simples, rápida de compilar e que resulta em código enxuto e eficiente
- Nascimento ligado ao sistema operacional UNIX

- Se espalhou nos microcomputadores dos anos 80
- Primeira padronização “oficial”: ANSI-C, de 1983
- Grande portabilidade em relação a outras linguagens, foi usada em todas as arquiteturas de processadores
- C++ é uma extensão do ANSI-C
 - contém recursos de orientação a objetos
 - é muito mais complexa do que C
- Última padronização: C11 (padrão ISO 9899:2011)

- Linguagem de médio nível
- Permite programação estruturada e modular
- Relativamente simples e portátil
- Produz código nativo eficiente
- Tem **legibilidade limitada** e **exige disciplina**
- Bastante poderosa mas possui poucos recursos de nível mais alto (sem OO)
- Extensão de arquivo: **.c**

Por que aprender C?



- Muito popular
- Linguagem madura e universal
- Possibilita desenvolvimento em muitos contextos
 - hardware embarcado, sistemas operacionais e também desktop
- Muito código legado existente
- É a base para o futuro aprendizado de C++
 - C e C++ são focados em **alto desempenho**
- Também influenciou a sintaxe de inúmeras linguagens

Tipos de dados

Tipos de dados

Valores numéricos



- Inteiro

- `int`: -8, 0, 12, 27354

- Real

- `float`: 1.0, 4.567, 0.28719
- usa o ponto decimal ao invés da vírgula

Tipos de dados

Valores lógicos, caracteres e strings



- Não existe tipo lógico (booleano)
 - usa-se o tipo `int`
 - falso é `0` e verdadeiro é `1`
- Caracter
 - `char`: contém apenas um caracter
 - usa aspas simples: `'a'`, `'1'`, `' '`
- *String*
 - usa aspas duplas: `"abc123"`, `"Isto é apenas um teste!"`, `""`
 - manipular *strings* é mais complicado que em Python

Primeiro programa



```
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

Compile e execute com:

```
$ gcc -Wall hello.c -o hello && ./hello
```

Saída e comentário

- O comando de saída é `printf(formato, parâmetros)`
- Exemplos:
 - `printf("Olá, mundo!");`
 - `printf("%i", 123);`
 - `printf("%f", 4.56);`
 - `printf("resultado: %i", 7 < 8);`
 - `printf("%c", 'z');`
 - `printf("Bom dia, %s!\n", "usuário");`

- Comentário de uma linha:
 - Começa com `//` e vai até o final da linha
- Comentário de uma ou várias linhas:
 - Começa com `/*` e termina com `*/`
- Exemplos:
 - `// Primeiro programa`
 - `printf("O total foi %i\n", total);`
 - `/* o cálculo a seguir será implementado depois:`
 - `total = soma + resto`
 - `*/`

Expressões

- Adição $+$, subtração $-$ e multiplicação $*$
 - $1 + 2$
 - $3 - 4.5$
 - $6.7 * 8.9$
- Mesmas regras de precedência que na Matemática:
 - $1 + 2 * 3 \rightarrow 7$
 - $(1 + 2) * 3 \rightarrow 9$
- Operador negativo $-$
 - $-(4 + 5 - 6)$

- Divisão fracionária /
 - $51.2 / 4 \rightarrow 12.8$
 - $7 / 2.0 \rightarrow 3.5$
 - quando pelo menos um dos operandos é `float`
- Divisão inteira (ou quociente) /
 - $7 / 2 \rightarrow 3$
 - quando os dois operandos são `int`
- Módulo (ou resto da divisão) %
 - $7 \% 2 \rightarrow 1$
 - $123 \% 100 \rightarrow 23$

Operadores de comparação (ou relacionais)

- Igualdade `==` e desigualdade `!=`
 - `3 * 5 == 15` → 1
- Maior `>` e maior ou igual `>=`
 - `12 > 18` → 0
 - `4 >= 4` → 1
- Menor `<` e menor ou igual `<=`
 - `4 * 5 < 30` → 1
 - `(4 - 2 * 8) <= (9 / 3 + 5)` → 1
 - Atenção com a precedência! Na dúvida, use parênteses...

- Operador “não”: !

- também chamado de **negação** ou **complemento**
- ! (3 == 3) → False equivale a (3 != 3)
- ! (4 >= 5) → True equivale a (4 < 5)
- ! (6 > 7) → True equivale a (6 <= 7)

- Tabela verdade:

- ! 0 → 1
- ! 1 → 0

- Operador “e”: **&&**
 - também chamado de **conjunção**
 - $(2 + 2 == 4) \ \&\& \ (3 < 5) \rightarrow 1$
- Tabela verdade:
 - $0 \ \&\& \ 0 \rightarrow 0$
 - $0 \ \&\& \ 1 \rightarrow 0$
 - $1 \ \&\& \ 0 \rightarrow 0$
 - $1 \ \&\& \ 1 \rightarrow 1$

- Operador “ou”: `||`
 - também chamado de **disjunção**
 - `(20 > 10) || (! 1) → 1`
- Tabela verdade:
 - `0 || 0 → 0`
 - `0 || 1 → 1`
 - `1 || 0 → 1`
 - `1 || 1 → 1`

Precedência dos operadores



- Do mais importante para o menos importante:
 - `()`
 - `-` (negativo)
 - `*` `/` `%`
 - `+` `-`
 - `==` `!=` `<` `<=` `>` `>=`
 - `!`
 - `&&`
 - `||`
- Na dúvida, use parênteses!

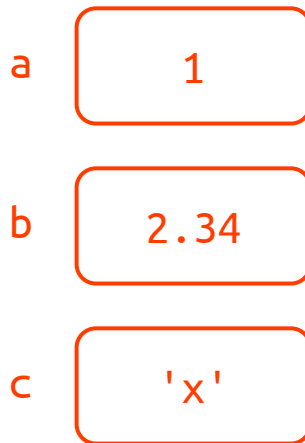
Práticas

- Calcule o valor das expressões:

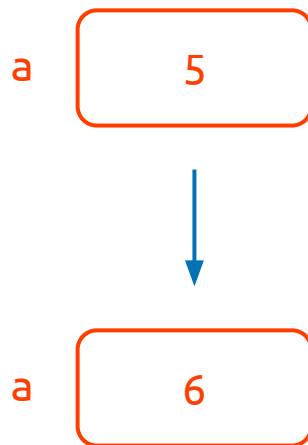
- $1 + 3 * (2 - 4 * 10 / 5) \rightarrow$
- $6 - 3 / 2 \rightarrow$
- $9.0 / 2 + 1 \rightarrow$
- $2 * 4 > 13 \rightarrow$
- $(5 > 2) \&\& (1 + 3 == 10 - 6) \rightarrow$

Variáveis

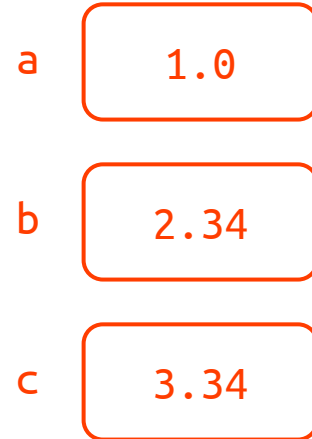
- **Variáveis** contém um único valor, de um certo tipo
 - inteiro, real, caracter ou *string*
 - este **tipo não pode ser mudado**
- O tipo precisa ser declarado junto com a primeira atribuição:
 - `int a = 1;`
 - `float b = 2.34;`
 - `char c = 'x';`



- O operador de **atribuição** = permite mudar o valor de uma variável já existente
 - não confunda com o operador de igualdade ==
- Exemplo:
 - `int a = 5;`
 - `...`
 - `a = 6;`



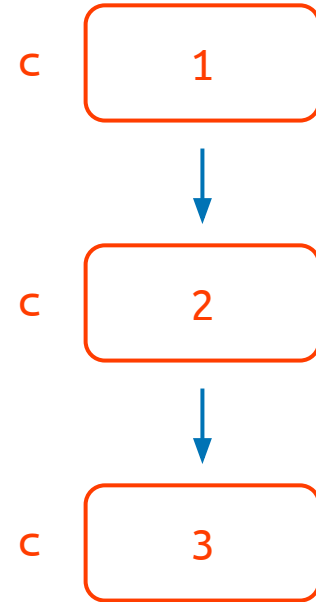
- É comum calcular uma expressão e então atribuir o seu resultado a uma variável
- Exemplo:
 - `float a = 1.0;`
 - `float b = 2.34;`
 - `float c = a + b;`



- Também é comum atualizar uma variável usando o seu valor anterior

- Exemplo:

- `int c = 1;`
- `...`
- `c = c + 1;`
- `...`
- `c++;`



Práticas

- Calcule a área A de um círculo de raio $r=2$, e apresente a medida na tela. Use uma variável auxiliar π . A fórmula da área é $A = \pi r^2$.
- Escolha alguma temperatura em graus Celsius e atribua à variável C . Apresente esta convertida em Fahrenheit, usando a fórmula de conversão $F = (9 * C + 160) / 5$.

Perguntas?